

Chapitre 4 Structures conditionnelles

Dans un programme, il peut être nécessaire de réaliser différentes instructions ou groupes d'instructions selon certaines conditions (exemple : si la température est négative alors faire ceci sinon faire cela). Pour cela, on utilise les structures conditionnelles, au nombre de deux (`if` et `switch`).

Structure `if (expression) {...} else {...}`

L'instruction conditionnelle `if (expression) {...} else {...}` est une conditionnelle à deux branches : si une condition est vraie, alors faire ceci, sinon faire cela.

Syntaxe

```
if (expression)
    instruction;

ou

if (expression)
    instruction1;
else
    instruction2;
```

Si `expression` est vraie, `instruction1` est exécutée. Sinon, c'est `instruction2` qui est exécutée.

Remarques

- `expression` doit être de type entier (booléen), c'est-à-dire valoir la valeur 0 (« faux ») ou toute valeur différente de 0 (considérée alors comme « vrai »).
- `instruction` peut regrouper plusieurs instructions placées entre les accolades de début et de fin de bloc : `{ instruction1; instruction2; ... ; }`
- Lorsque plusieurs instructions `if` sont imbriquées, `else` se réfère au dernier `if`.

Exemple

```
if (nScore1 == nScore2)
{
    cout << "Match nul." << endl
}
else
{
    if (nScore1 > nScore2)
    {
        cout << "L'équipe 1 a gagné" << endl;
    }
    else
    {
        cout << "L'équipe 2 a gagné" << endl;
    }
}
```

Opérateurs de comparaison

Les opérateurs de comparaison sont souvent utilisés pour comparer des valeurs entre elles afin d'évaluer des expressions.

Valeur v_1	valeur v_2	$v_1 < v_2 (<)$	$v_1 \leq v_2 (\leq)$	$v_1 > v_2 (>)$	$v_1 \geq v_2 (\geq)$	$v_1 == v_2 (=)$	$v_1 != v_2 (\neq)$
8	3	0 (F)	0 (F)	1 (V)	1 (V)	0 (F)	1 (V)
8	8	0 (F)	1 (V)	0 (F)	1 (V)	1 (V)	0 (F)
3	3	0 (F)	1 (V)	0 (F)	1 (V)	1 (V)	0 (F)
3	8	1 (V)	1 (V)	0 (F)	0 (F)	0 (F)	1 (V)

Tableau 6 : Comparaisons de valeurs. Les valeurs v_1 et v_2 sont données à titre d'exemple pour expliquer les résultats (V $\equiv$ VRAI, F $\equiv$ FAUX).



Faire très attention à bien utiliser l'opérateur de comparaison `==` (double signe d'égalité) et non l'opérateur d'affectation `=` (simple signe d'égalité) pour vérifier l'égalité de deux valeurs. Sauf si b est égal à 0, $(a = b)$ est toujours vrai !

Combinaisons de booléens

L'instruction `if` peut combiner des booléens entre eux (ou, et, ou exclusif, non, ...) ; Le Tableau 5 page 21 indique les opérations et les résultats de combinaisons de booléens.

Lorsqu'on combine plusieurs expressions entre elles, il est préférable d'entourer par des parenthèses chaque expression évaluée : `if ( (a==b) || (c != d) )`. C'est plus lisible et cela lève des ambiguïtés dans les évaluations par certains compilateurs.

Instruction `switch`

L'instruction `switch` peut être vue comme une instruction `if` avec plusieurs branches. C'est une instruction multidirectionnelle utilisée pour gérer plusieurs décisions. Dans ce cas, `switch` génère un code plus lisible que le `if ... else` et s'exécute parfois plus rapidement que le `if ... else`.

Syntaxe

```
switch (expression)
{
    case valeur1 : instruction1; break;
    case valeur2 : instruction2; break;
    case valeur3 : instruction3; break;
    ...
    Default : instruction;
}
```

Remarques

- `expression` doit être de type entier (booléen), c'est-à-dire valoir la valeur 0 (« faux ») ou toute valeur différente de 0 (considérée alors comme « vrai »). `expression` peut être un caractère (car les caractères sont des entiers), mais ne peut représenter un réel ou une chaîne de caractères.
- `valeur1`, `valeur2`, etc. doivent être de type entier (y compris `char`) ou représenter une expression qui donne une valeur entière. Chaque `valeur` doit représenter une seule valeur (les valeurs multiples ne sont pas autorisées) et doit être unique.
- L'instruction de chaque branche peut regrouper plusieurs instructions placées entre les accolades de début et de fin de bloc : `{ instruction1; instruction2; ... ; }`. Dans

ce cas, les accolades de bloc peuvent être omises car les "case" délimitent chaque branche. Il est néanmoins vivement conseillé de conserver ces accolades pour faciliter la lecture des programmes.

- L'évaluation de l'expression a lieu dans l'ordre des "case".
- Dès que la valeur de l'expression correspond à la valeur qui suit un "case", toutes les instructions qui suivent ce "case" sont exécutées (même celles des autres "case" qui ne sont dans ce cas pas évalués). Pour provoquer une sortie du switch sans exécuter les instructions des autres "case", on utilise l'instruction `break` (voir Chapitre sur les structures itératives).
- L'instruction qui suit le "default" est exécutée si aucun branchement d'un "case" n'est réalisé.

Exemple 1

```
#include< stdio.h>
int main(void)
{
    char cCh = ' ';
    cout << "Entrez un caract\212re : "; cin >> cCh;

    switch (cCh)
    {
        case 'a' : cout << "Lettre a"; break;
        case 'b' : cout << "Lettre b"; break;
        default : cout << "Autre caract\208re";
    }
}
```

Exemple 2

```
#include< stdio.h>
int main(void)
{
    int nJour = 0;
    cout << "Entrez le num\202ro du jour : "; cin >> nJour ;

    switch(nJour)
    {
        case 1 : cout << "Lundi";    break;
        case 2 : cout << "Mardi";    break;
        case 3 : cout << "Mercredi"; break;
        case 4 : cout << "Jeudi";    break;
        case 5 : cout << "Vendredi"; break;
        case 6 : cout << "Samedi";   break;
        case 7 : cout << "Dimanche"; break;
        default: cout << "jour invalide";
    }
}
```